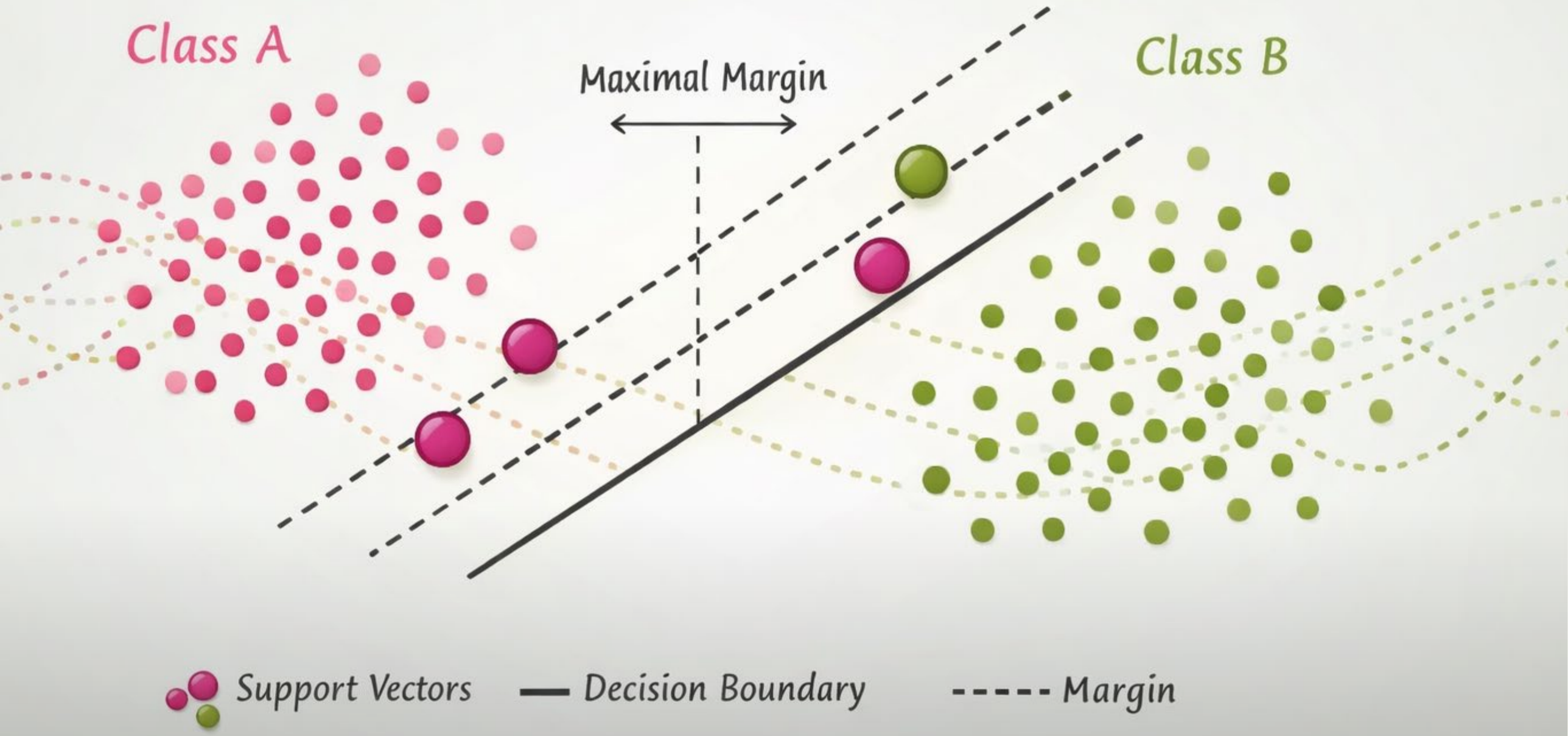


SUPPORT VECTOR MACHINES

INTRODUCTION TO SVM

- Support vector machines (SVMs) are an approach developed by computer scientists in the 1990s
    - Do not directly provide probability estimates
- Perform well on a range of problems
    - May be less widely used following advances in artificial neural networks
    - Remain mathematically instructive
- We begin by reviewing the definition of a hyperplane

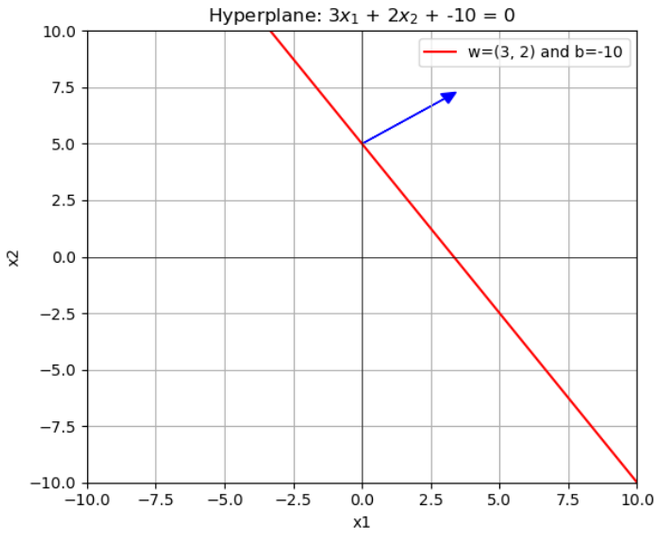
HYPERPLANES

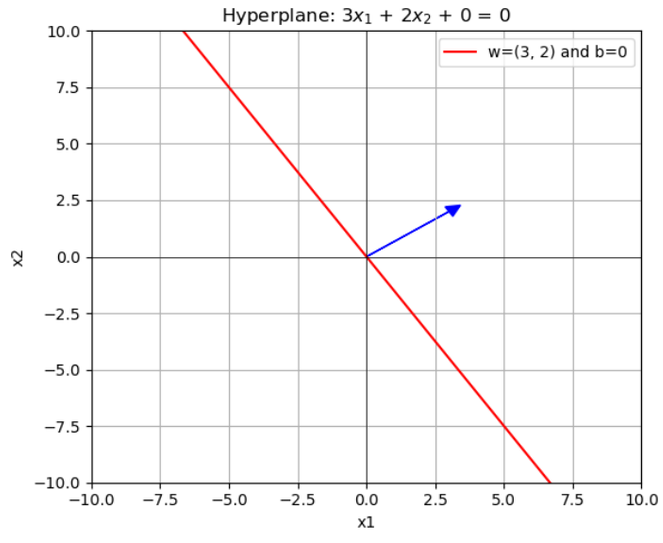
- An affine space is a "flat" set of points, such as a line or plane, that need not pass through the origin.
- A hyperplane is a generalization of a line in p dimensions
- Formally, a hyperplane in p dimensions is an affine subspace of dimension $p - 1$.
  - Its general equation is:

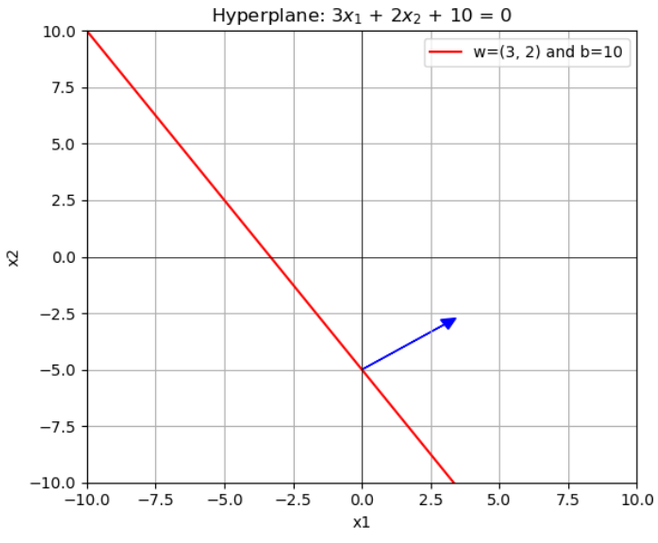
$$\mathbf{w} \cdot \mathbf{x} + b = 0$$

    - Where $\mathbf{w}$ is the normal vector (perpendicular to the hyperplane) that determines the orientation
    - b is the bias term
    - $\mathbf{x}$ is any point in the space
- When $p = 2$ (there are two features) a hyperplane is a line

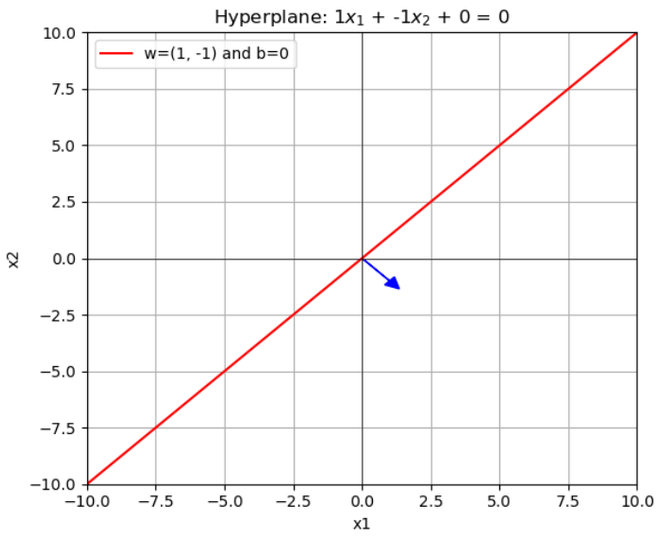
Changing b

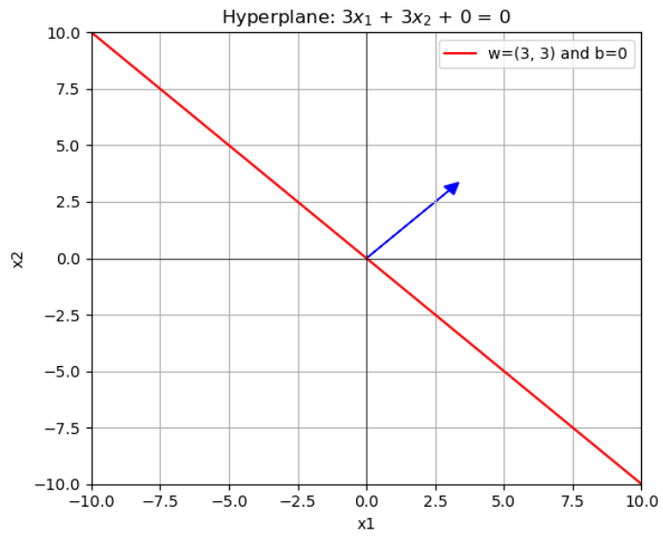


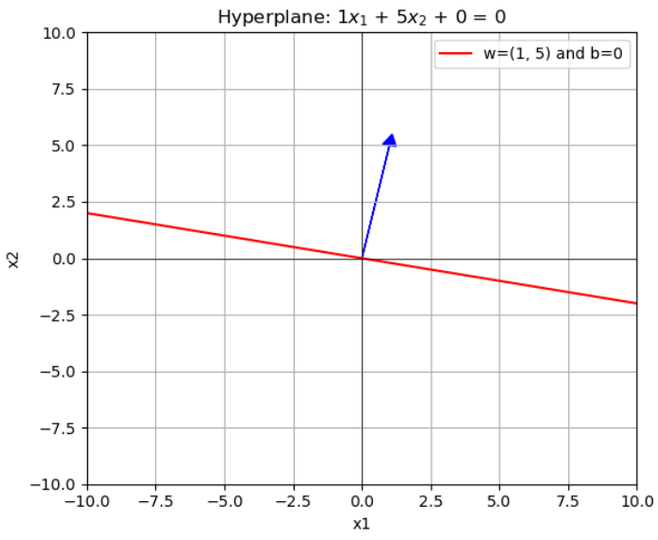




Changing w

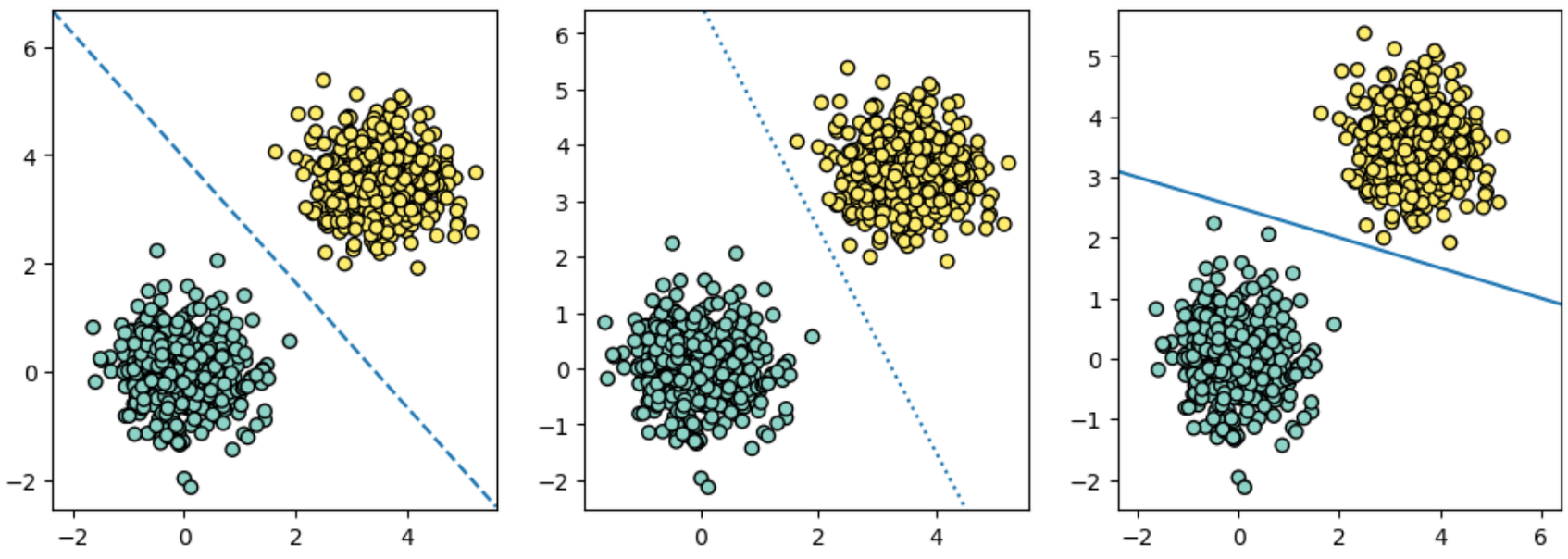






SVM: CHOOSING A SEPARATING BOUNDARY

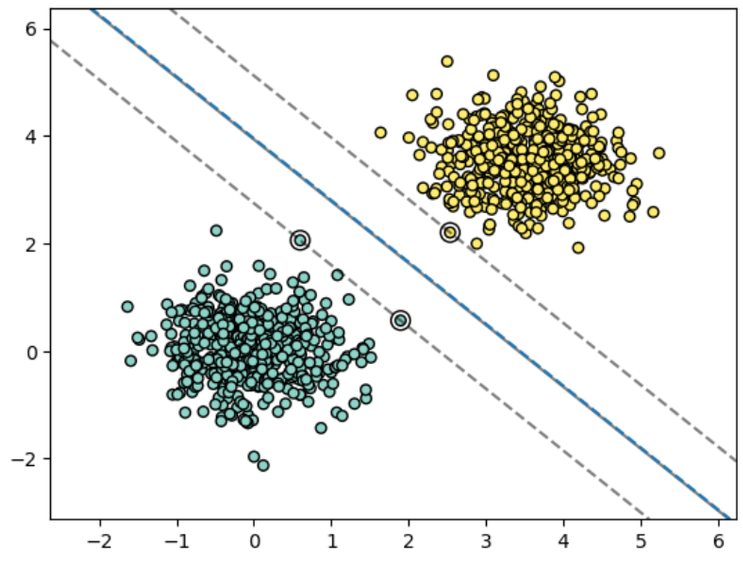
- SVM uses a hyperplane as a boundary between two classes.
  - With two features, this boundary is a line.
- These classes are linearly separable: each line below separates all training points correctly.
- Several boundaries work. Which line seems most optimal?



MAXIMAL MARGIN CLASSIFIER

- The decision boundary is the hyperplane that is farthest from the training observations
- Fits the “widest possible street”
- Notice that only the points on the edge of the street matter
- These are called support vectors (decision boundary is supported by these instances)
- Consider the maximal margin classifier for linearly separable classes.
- A training observation is a support vector if and only if its perpendicular distance to the decision boundary equals the smallest such distance among all training observations.

The support vectors are just points in this two dimensional feature space, but in higher dimensions, they will be actual vectors.



PERFECT SEPARATION AND ITS EXTENSIONS

- When perfect separation is possible, SVM chooses the boundary with the widest margin.
  - The margin is the distance from the boundary to the nearest training points.
- When a hyperplane cannot separate the classes perfectly, we can:
  1. Allow some points to cross the margin or the boundary.
  2. Expand the feature space to make separation easier.
- In the perfectly separable case: the maximal-margin (hard-margin) classifier.
- Otherwise, a soft-margin classifier is used

VECTORS, LENGTH, AND UNIT DIRECTIONS

- The L2 norm ‖w‖ is the usual Euclidean length of a vector.

  $$\|w\| = \sqrt{(w_1^2 + \cdots + w_p^2)}$$

- A **unit vector** has length 1. Divide a nonzero vector by its length to keep only its direction.

  $w = (3, 4) \quad \|w\| = 5 \quad u = w / \|w\| = (3/5, 4/5)$

DOT PRODUCTS

- Multiply matching coordinates, then add. The result is a number.

  $w \cdot x = w_1x_1 + \cdots + w_px_p = w^Tx = w'x$

  For $w = (3, 4)$ and $x = (2, 1)$: $w \cdot x = 3(2) + 4(1) = 10$

- If θ is the angle between the vectors, the dot product measures their alignment.

  $w \cdot x = \|w\| \, \|x\| \cos \theta$

- Interpretation:
  - Positive: angle below 90°. Zero: perpendicular. Negative: angle above 90°.

    Check: $(3, 4) \cdot (-4, 3) = -12 + 12 = 0$

PROJECTIONS

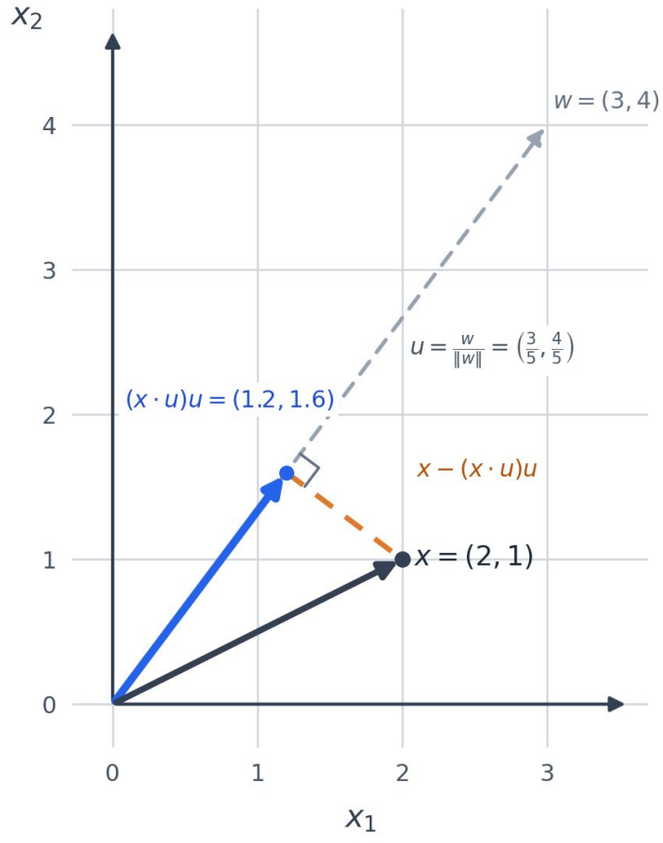
For $w = (3, 4)$ and $x = (2, 1)$: $w \cdot x = 3(2) + 4(1) = 10$

- Projecting x onto a direction gives its "shadow" along that direction.
- Use the unit direction $u = w / \|w\|$.

  Signed length: $x \cdot u = (w \cdot x) / \|w\|$

  Example: $10 / 5 = 2$
- The projected vector is $(x \cdot u)u$.

  $2(3/5, 4/5) = (1.2, 1.6)$
- The remainder is perpendicular to w.
- For hyperplane distance, project $x - x_0$ onto the normal direction u, where x_0 lies on the boundary.



- The projection and perpendicular remainder add back to x.

DISTANCE TO A SEPARATING HYPERPLANE

- Decision boundary (the street's centerline): $w \cdot x + b = 0$.
- x is the point to measure. x_0 is any point on the decision boundary, so $w \cdot x_0 = -b$.
- Project $x - x_0$ onto the unit normal $u = w / \|w\|$ to get the signed distance d.

  $$d = (x - x_0) \cdot u = (w \cdot x - w \cdot x_0) / \|w\| = (w \cdot x + b) / \|w\|$$

- The sign tells which side x lies on. The absolute value $|d|$ gives the shortest distance to the boundary.
  - Example: $w = (3, 4)$, $b = -5$, $x = (2, 1)$, $\|w\| = 5$
  - Score $= w \cdot x + b = 3(2) + 4(1) - 5 = 5$
  - Signed distance: $d = 5 / 5 = +1$ Shortest distance: $|d| = 1$
- Thus x lies 1 unit from the decision boundary, on the positive side. Dividing the score by $\|w\|$ converts it to distance.

MARGINS

- The margin is the distance from the decision boundary to the nearest training points.
- For a **correctly classified** point $\mathbf{x_i}$ with label y_i (± 1), its distance to the decision boundary is:

$$\frac{|\mathbf{w} \cdot \mathbf{x_i} + b|}{||w||} = \frac{y_i(\mathbf{w} \cdot \mathbf{x_i} + b)}{||w||}$$

- Scaling both w and b by the same positive constant preserves the boundary and distance
- The margin is then $\frac{1}{||\mathbf{w}||}$. Maximizing it is equivalent to minimizing:

$$\frac{1}{2}||\mathbf{w}||^2 = \frac{1}{2}\mathbf{w}'\mathbf{w}, \text{ subject to } y_{\mathrm{i}}(w \cdot x_{\mathrm{i}} + b) \geq 1 \text{ for every i.}$$

LABELS, SCALE, AND THE SVM OBJECTIVE

- Given labels $y_i = +1$ or -1:
  - $y_i(w \cdot x_i + b) > 0$ means correct classification.
- Multiplying both w and b by the same positive constant keeps the boundary and distances unchanged.
- For separable data, choose the scale so the smallest label-adjusted score is 1.

  $y_i(w \cdot x_i + b) \geq 1$ for every training point
- Now the nearest points have distance $1 / \|w\|$. A wider margin requires a smaller norm.

  Minimize $\frac{1}{2}\|w\|^2 = \frac{1}{2}w^T w$, subject to the constraints above.

SVM OPTIMIZATION PROBLEM

Training data: $\{(x_i, y_i)\}_{i=1}^{n} \quad \mathbf{x}_i \in \mathbb{R}^d, \qquad y_i \in \{-1, +1\}$

$$\min_{\mathbf{w},b} \ \frac{1}{2}\|\mathbf{w}\|^2$$

$$\text{s.t.} \ \ y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1, \qquad i = 1, \ldots, n$$

SVM DECISION PROCESS

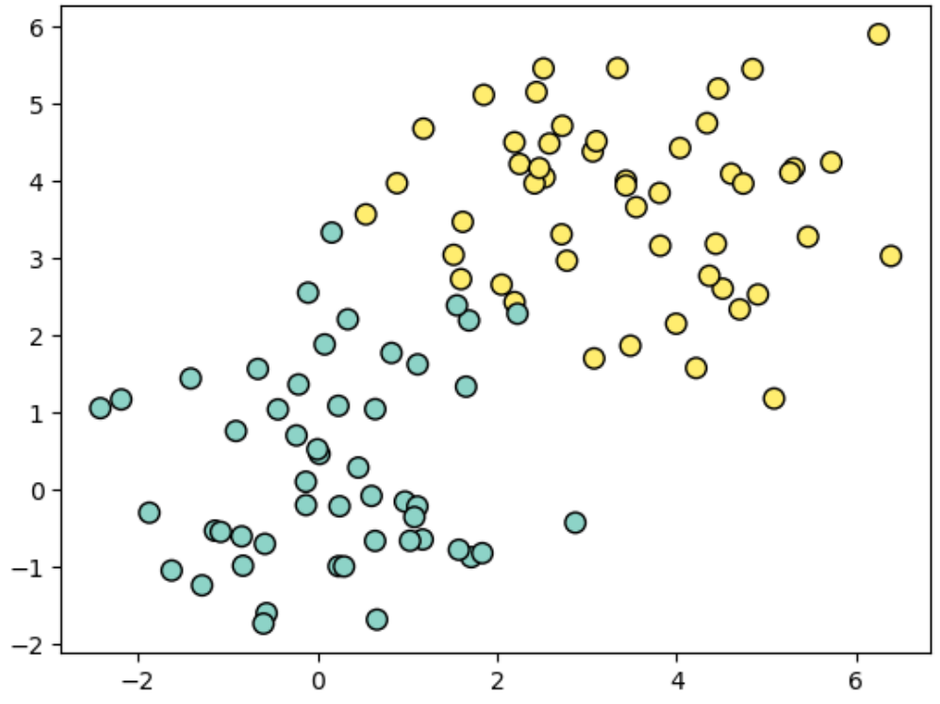
- **After training**, we obtain:
- Weight vector **w**
- Intercept **b**
- For a new data point **x**, compute the **decision score**:

$$f(x) = w \cdot x + b$$

- The prediction is based on the **sign**:
- If $f(x) > 0 \rightarrow$ predict **+1**
- If $f(x) < 0 \rightarrow$ predict **−1**
- Geometric interpretation:
- The hyperplane $w \cdot x + b = 0$ is the decision boundary
- The sign tells which **side of the boundary** the point lies on
- $|f(x)|$ is proportional to the **distance from the boundary**
- Larger magnitude → more confident prediction

POTENTIAL PROBLEMS

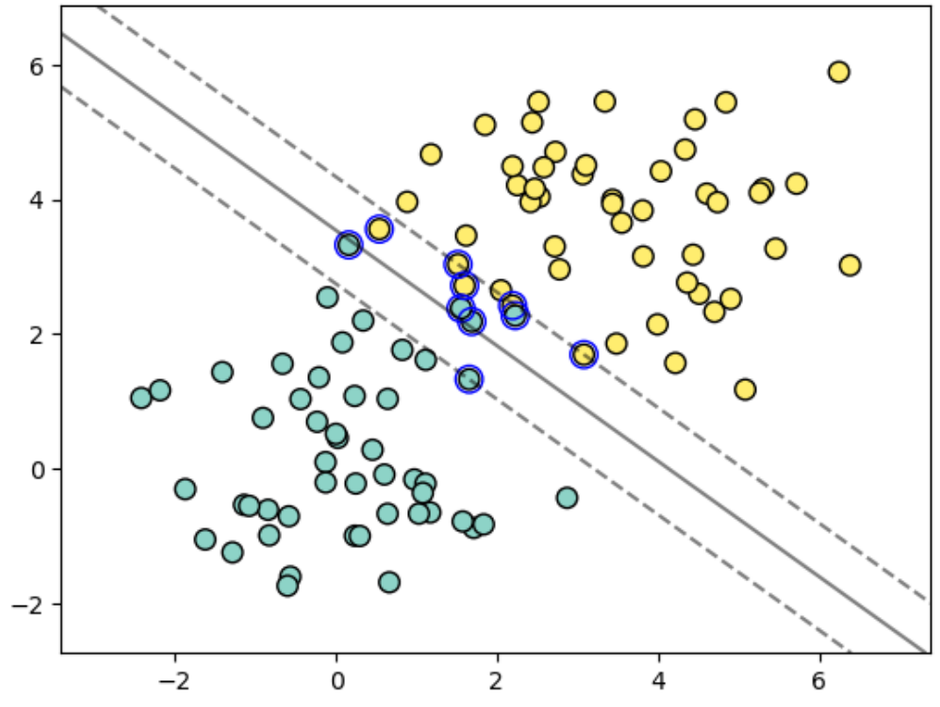
- What if a separating hyperplane does not exist?
- Then there is no maximum marginal classifier



SOFT MARGIN SVM

POTENTIAL PROBLEMS

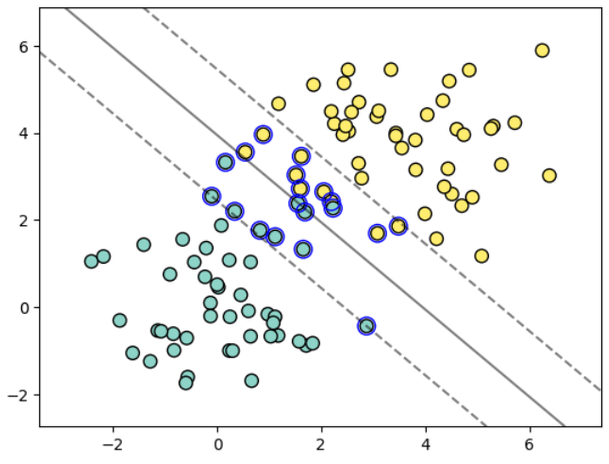
- In this case, a natural generalization would be to allow some points to spill into the “street” or margin



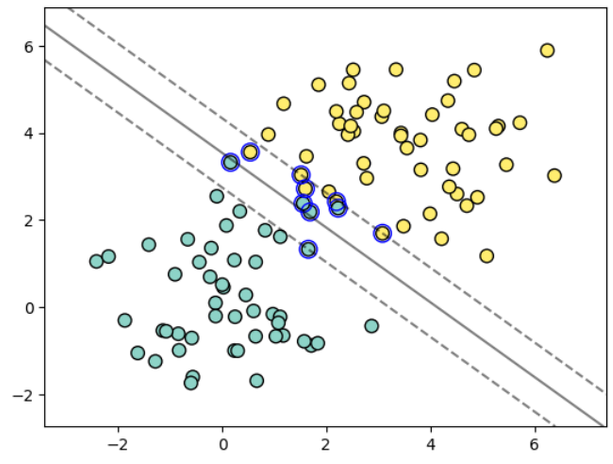
SUPPORT VECTOR CLASSIFIER (SOFT MARGIN)

- Hyper-parameter **C** controls the width of the road and the amount of over-spill that is allowed
- When **C** is small, the margin is wider and more points are involved in determining the direction of the boundary

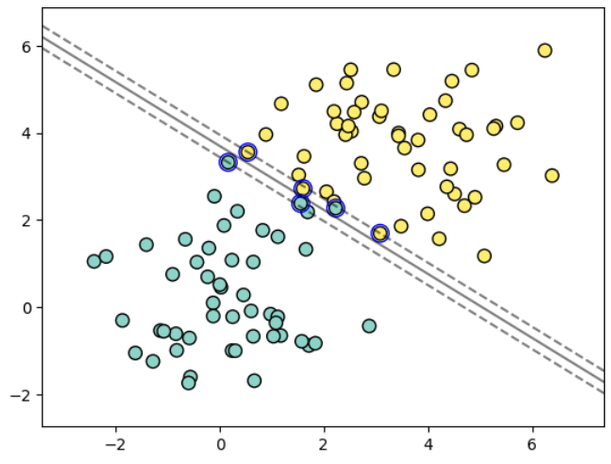
C=0.1



C=1



C=10



SLACK VARIABLES

- Let $f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b$. The margin edges satisfy $f(\mathbf{x}) = \pm 1$.
- At the soft-margin optimum ($C > 0$), $\xi_i = \max(0, 1 - y_i f(\mathbf{x}_i))$. This is the hinge loss.
- $\xi_i = 0$: on or beyond the correct margin edge.
- $0 < \xi_i < 1$: inside the margin, but correctly classified.
- $\xi_i = 1$: on the decision boundary.
- $\xi_i > 1$: on the wrong side of the boundary (misclassified).
- For $\xi_i > 0$ and $\mathbf{w} \neq 0$, the distance to the correct margin edge is $\frac{\xi_i}{\|\mathbf{w}\|}$. Slack itself uses score units, not distance units.

OPTIMIZATION W/ SLACK VARIABLES

C > 0 sets the cost of a margin violation.

$$\textbf{Training data:} \quad \{(\mathbf{x}_i, y_i)\}_{i=1}^{n}, \quad \mathbf{x}_i \in \mathbb{R}^d,\ y_i \in \{-1, +1\}$$

$$\begin{aligned} \min_{\mathbf{w},b,\{\xi_i\}} \quad & \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{n}\xi_i \\ \text{s.t.} \quad & y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i, \quad i = 1, \ldots, n \\ & \xi_i \geq 0, \quad i = 1, \ldots, n \end{aligned}$$

The first term favors a wider margin: total street width = $2/\|\mathbf{w}\|$.
The second term charges C for each unit of slack.

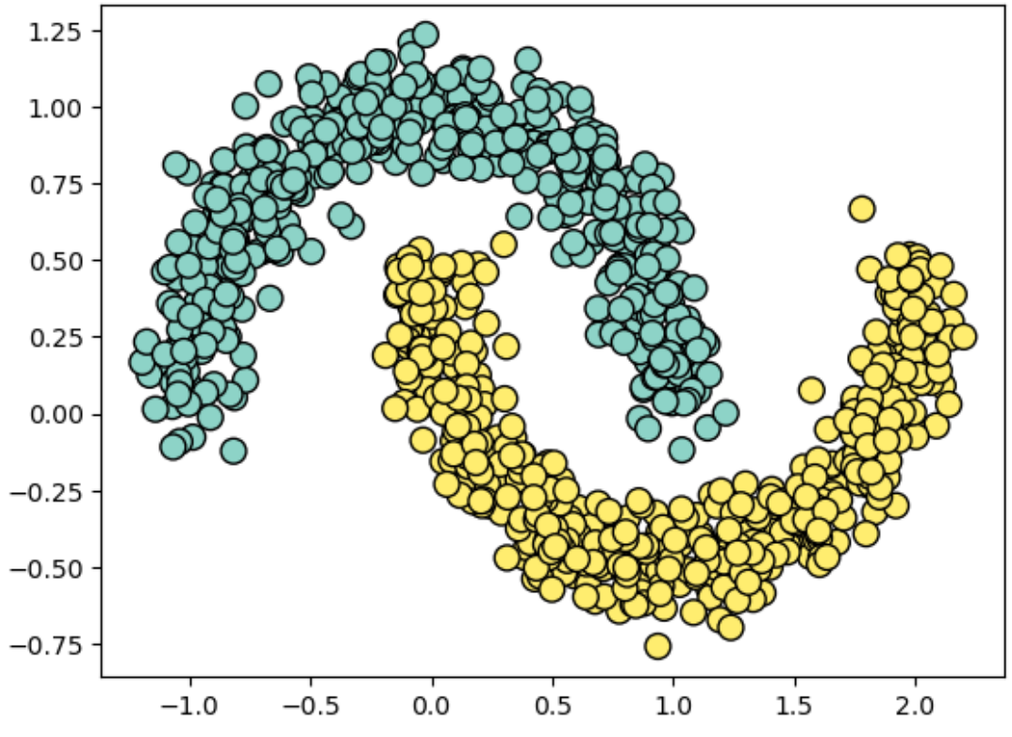
SUPPORT VECTORS

- The points inside the margin (or points wrongly misclassified, if any) are the support vectors
  - Only these points effect the decision boundary
    (the support vectors influence the position and orientation of the hyperplane)
  - When C is small, the street is wider so more points are involved in making the decision boundary
    - Can lead to underfitting
  - When C is large, the street more narrow so fewer points are involved in making the decision boundary
    - Can lead to overfitting

NON-LINEARLY SEPARABLE DATA

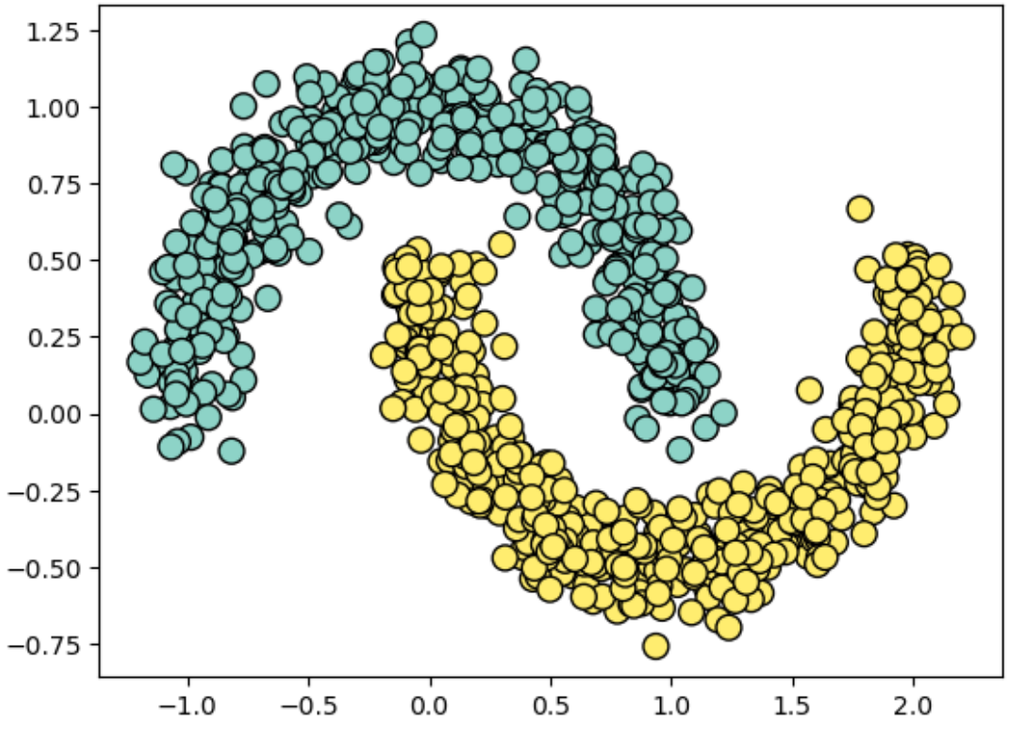
NON-LINEARLY SEPARABLE

- Now what happens if the classes aren't linearly separable



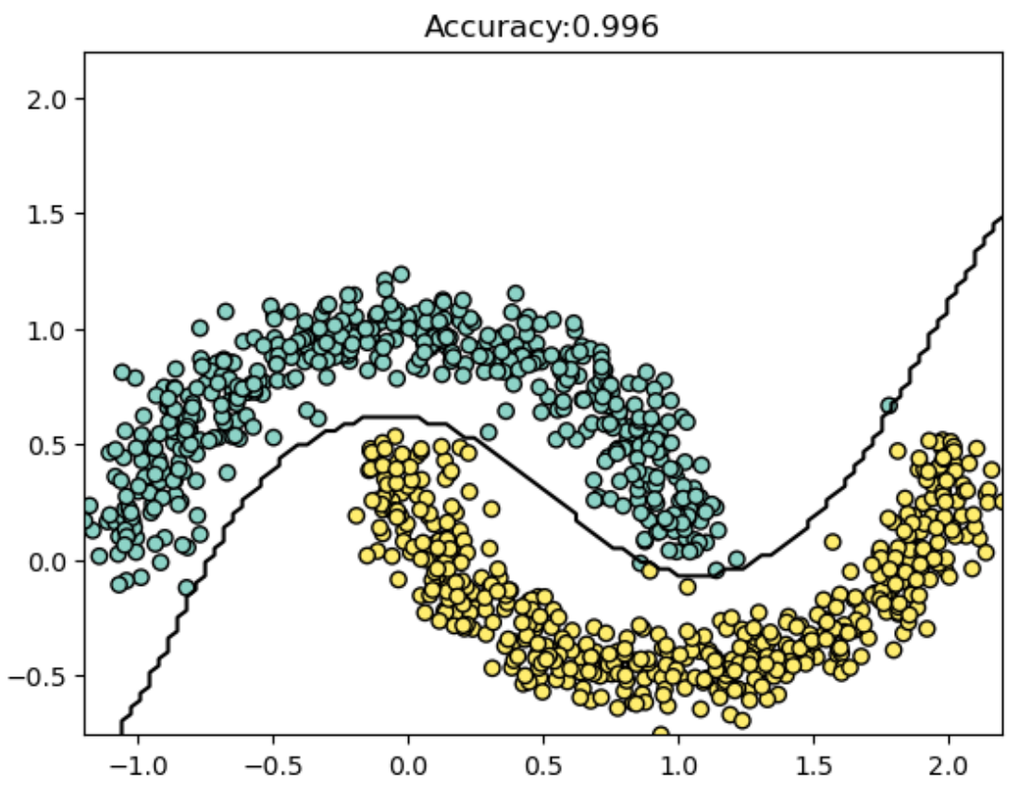
NON-LINEARLY SEPARABLE

- One possibility is to add polynomials terms to a linear model (e.g. logistic regression)



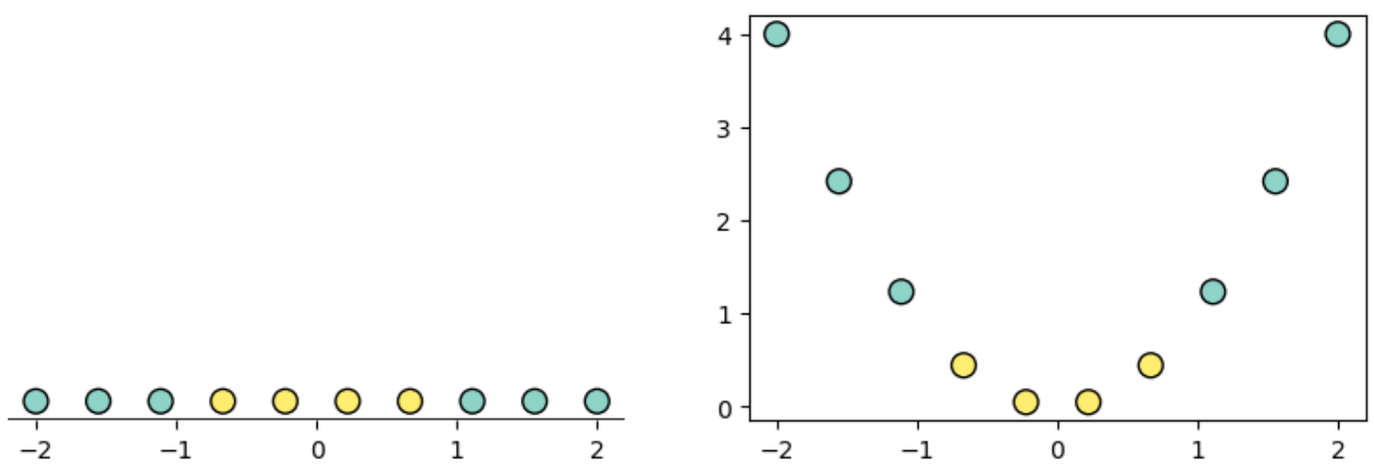
NON-LINEARLY SEPARABLE

- This is the decision boundary of a logistic regression model after adding 4th order polynomials to the feature matrix



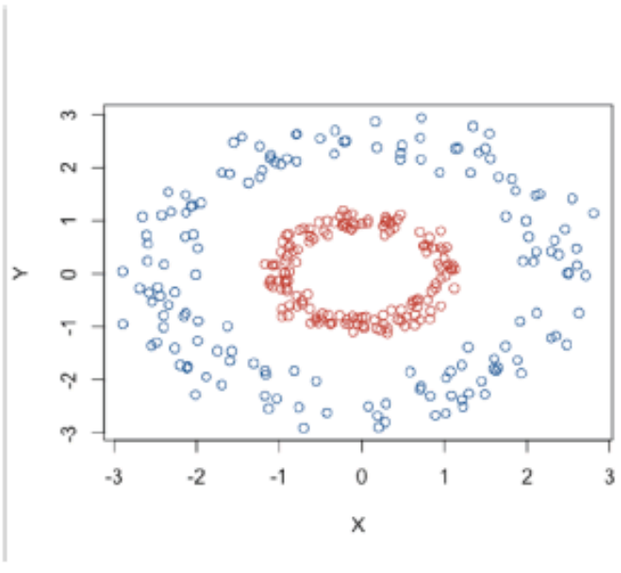
EXAMPLES

- Here we can visualize how adding higher dimensions can create linear separability
  - In 1-dimension the data is not linearly separable
  - After applying the transformation $\phi(x) = x^2$ we have linear separation

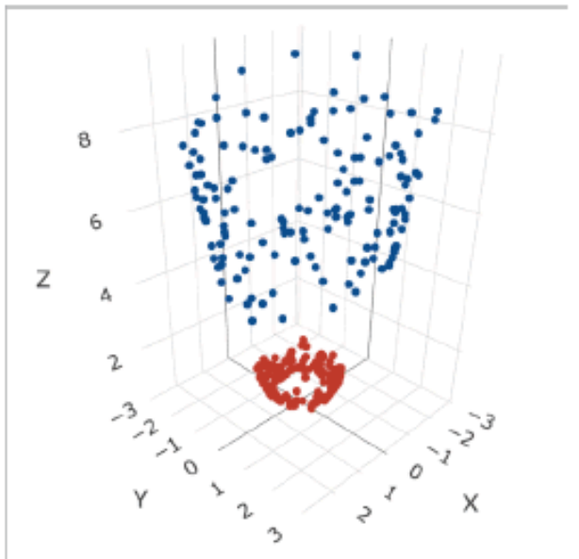


ANOTHER EXAMPLE

Another Example



(a) Prior to application of $\phi()$



(b) Following application of $\phi()$

Figure 5: The 'Kernel Trick': Transforming data to higher dimensions to gain linear separability [21].

A SOLUTION AND A PROBLEM

- While mapping to higher-dimensional space can create linearly separable data…
- …It can be computationally expensive to directly compute the transformation
  - E.g., polynomial space gets really big really fast, especially with lots of features
- …It is also hard to determine how many higher-dimensions to include
- This is where SVMs and the **Kernel Trick** come to the rescue

SUPPORT VECTOR MACHINES

- The main idea behind SV*Machines* is to
  - create linear separation by expanding the feature space
  - but in a specific way using **kernels**
    - a computationally efficient trick for expanding the feature space
  - this is called the **kernel trick**

DOT PRODUCTS IN THE DUAL

- $\phi(x_i)$A feature map φ(x) can make a nonlinear boundary linear in feature space.
- Explicitly constructing φ(x) may require many features.
- The SVM dual, introduced next, uses training data only through pairwise dot products:
- $x_i^T x_j$ in the original space
- $\varphi(x_i)^T \varphi(x_j)$ in the transformed space
- A kernel computes the second quantity directly from x_i and x_j.

THE KERNEL TRICK

- In machine learning, a **kernel function**, $K(x_i, x_j)$, is a function capable of computing the dot product between two transformed vectors based only on the original vectors **without having to compute the transformation** ϕ

$$K(x_i, x_j) = \phi(x_i)^\top \phi(x_j)$$

- It computes the dot product in feature space **without explicitly computing** ϕ.
- This is the kernel trick.

COMMON KERNELS

- Linear: $K(a, b) = a^T b$
- A straight hyperplane in the original feature space.
- Polynomial: $K(a, b) = (\gamma a^T b + r)^d$
- Adds feature interactions. Larger degree d permits more complex global boundaries.
- Gaussian RBF: $K(a, b) = \exp(-\gamma \|a - b\|^2)$
- Nearby points have greater similarity. Larger γ makes each point's influence more local.
- The kernel changes pairwise similarities. C still sets the penalty for margin violations.
- Scale features first. Tune C and kernel parameters together using validation data.

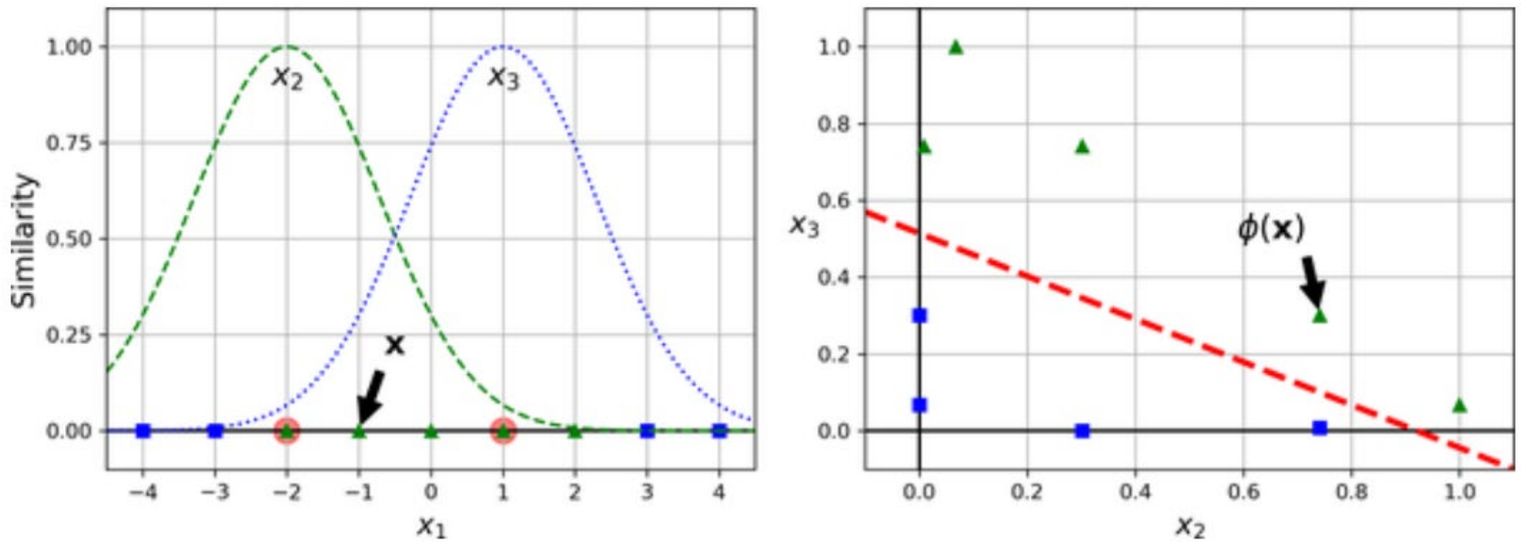
THE RBF KERNEL

- The Gaussian RBF kernel is very popular and is the default for the **`SVC`** and **`SVR`** functions in sklearn
- $K(\mathbf{a}, \mathbf{b}) = \exp(-\gamma(||\mathbf{a} - \mathbf{b}||^2)$

MORE ABOUT THE RBF KERNEL

- The γ parameter is sort of the inverse standard deviation of the Gaussian used to compute the similarity features
- As γ decreases, the standard deviation gets larger and it is easier to have a non-negligible distance from the landmark
    - Small γ will smooth more and can lead to underfitting
- As γ increases, the standard deviations get smaller and it is many of the distances from the landmark will be very close to zero
    - Large γ will capture small variations and can lead to overfitting

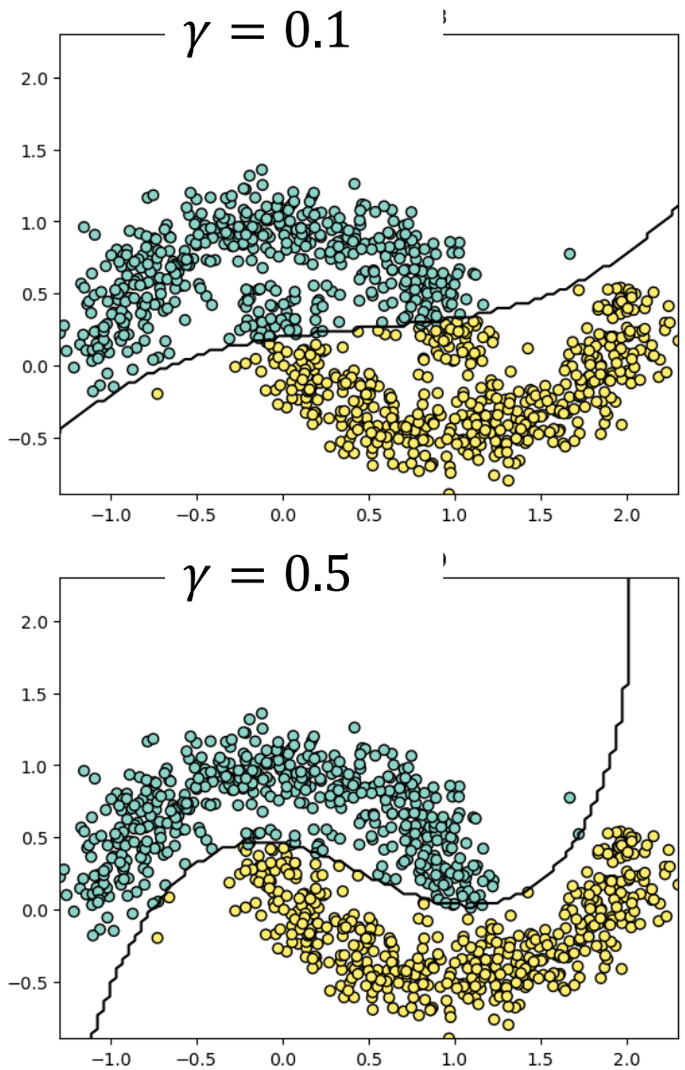
EXAMPLE OF RBF FEATURES

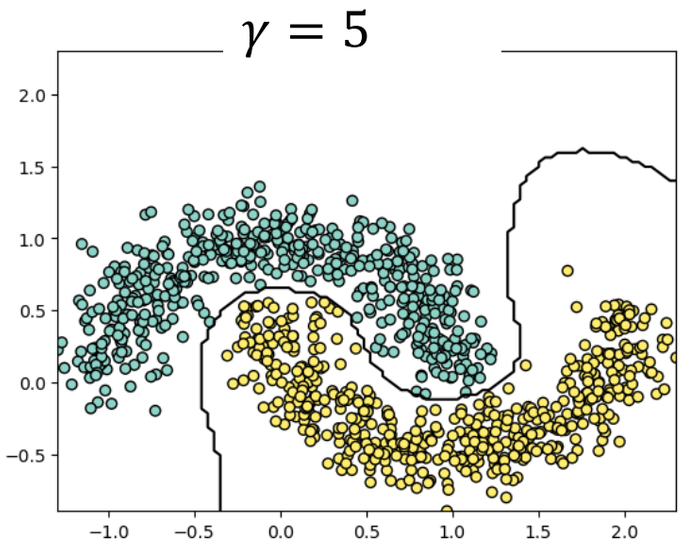


- Example where original features space is 1-dimensional (just x_1)
- Let x_2 be the similarity of x_1 to landmark 1 and x_3 be the similarity of x_1 to landmark 2
- For example, if $\gamma = 0.3$, for $x_1 = -1$

$$x_2 = \exp(-0.3 * 1^2) \approx 0.74$$
$$x_3 = \exp(-0.3 * (-2)^2) \approx 0.30$$

COMPARING γ





GAUSSIAN RBF KERNEL

- This is a very popular kernel and is the default for the **`SVC`** function in sklearn
- As γ decreases, the standard deviation gets larger and it is easier to have a non-negligible distance from the landmark
    - Small γ will smooth more and can lead to underfitting
- As γ increases, the standard deviations get smaller and it is many of the distances from the landmark will be very close to zero
    - Large γ will capture small variations and can lead to overfitting

SVM REGRESSION

SVR AND E-INSENSITIVE LOSS

- For regression, predict a numeric response:
- $f(x) = w^T x + b$
- The ε-tube contains points with $|y_i - f(x_i)| \leq \varepsilon$. Its vertical width is 2ε in response units.
- The loss is $L\varepsilon(y_i, f(x_i)) = \max(0, |y_i - f(x_i)| - \varepsilon)$. Errors inside the tube incur zero loss.
- Example: if $\varepsilon = 0.5$ and the absolute error is 0.8, the loss is 0.3. An absolute error of 0.2 gives zero loss.
- Classification penalizes a margin shortfall. SVR penalizes the amount by which a residual exceeds ε.

SVR OPTIMIZATION

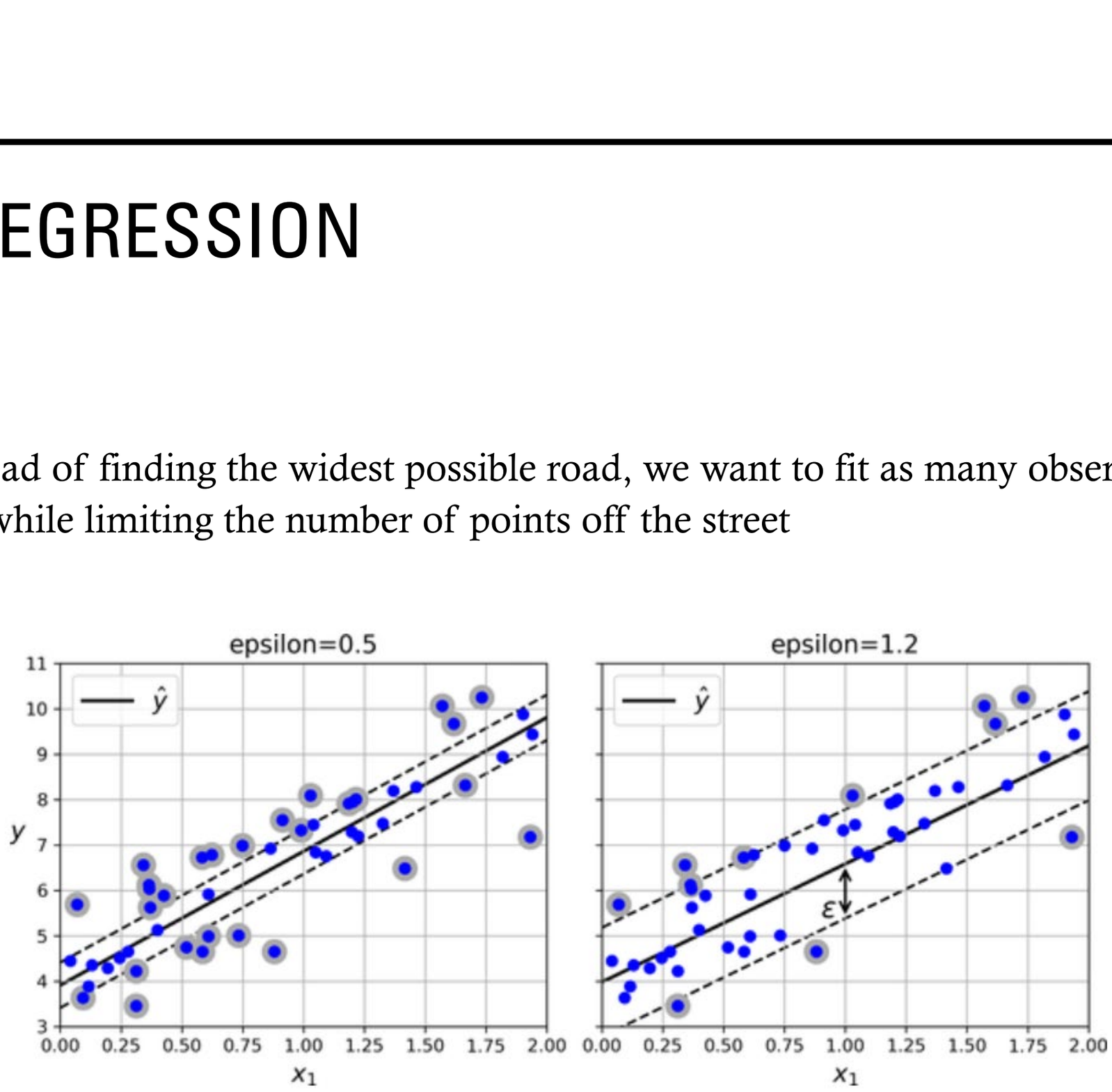
$$\textbf{Training data:} \quad \{(\mathbf{x}_i, y_i)\}_{i=1}^{n}, \quad \mathbf{x}_i \in \mathbb{R}^d,\ y_i \in \mathbb{R}$$

$$\begin{aligned}
\min_{\mathbf{w},b,\{\xi_i,\xi_i^*\}} \quad & \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{n}(\xi_i + \xi_i^*) \\
\text{s.t.} \quad & y_i - (\mathbf{w}^\top \mathbf{x}_i + b) \le \varepsilon + \xi_i, \quad i = 1,\dots,n \\
& (\mathbf{w}^\top \mathbf{x}_i + b) - y_i \le \varepsilon + \xi_i^*, \quad i = 1,\dots,n \\
& \xi_i \ge 0, \quad \xi_i^* \ge 0, \quad i = 1,\dots,n
\end{aligned}$$

ξ_i and ξ_i^* measure excess above and below the ε-tube.
C penalizes excess residual size, rather than counting outliers.

SVM REGRESSION

- Here, instead of finding the widest possible road, we want to fit as many observations on the street while limiting the number of points off the street



SVM REGRESSION

- SVR support vectors lie on or outside the ε-tube. Their nonzero dual weights contribute to the predictions.
- If more points are added inside the margin, predictions don't change (this is referred to as being ϵ-insensitive)

SVR PREDICTIONS

- Linear SVR predicts the number $f(x) = w^T x + b$. There is no sign threshold.
- Kernel SVR uses the same kernel in fitting and prediction. Each support vector contributes a signed amount.
- The two multipliers α_i and α_i^* correspond to the two sides of the ε-tube. Their difference is the prediction weight.

$$f(x) = \sum_{i=1}^{n} (\alpha_i - \alpha_i^*)\, K(x_i, x) + b$$

Only nonzero differences contribute. The output stays in response units.

SUMMARY

- SVMs are a very different way of creating a classifier than other methods that we've looked at
  - No notion of finding $P(y_i|X_i)$
- Only binary classifier
  - Can be extended to multiclass problems using one-versus-one or one-versus-all strategies
- Pretty slow for large datasets